Executive Summary

The AI Failures Museum is an education web application that we have designed to help people learn from real-world AI system failures. Rather than showcasing successful AI deployments, we have focused on where, how and why AI supported systems fail. By studying these failures, users can develop their own thoughts about the limitations of AI and then recognise potential warning signs in their own projects. Understanding the importance of proper validation and accountability.

What the prototype currently achieves:

- Content library: The system contains 11 curated exhibits covering diverse domains including environmental management, agricultural surveillance, fintech and lending, irrigation management, public health and smart city monitoring, autonomous transportation, conservation biology, real estate and legal tech, logistics and healthcare, and civil engineering infrastructure.

- Browse functionality: Users can view a list of all exhibits through our home page, which includes category filtering tabs (All Cases, Healthcare, Tech, Social Media, etc.) and severity badges (critical, high, medium) for each case. Clicking on any exhibit card takes users to a detailed page about the exhibit.

-Interactive quizzes: The system includes reflective quiz questions that test the user and encourage them to learn and understand the content. Users can complete multiple-choice quizzes where answer options are randomised to prevent memorisation. After submission, users immediately see their score, color-coded feedback (green for correct, red for incorrect), and detailed explanations for each question. Users can retake quizzes as many times as they want to improve understanding.

- Comments and discussion: Visitors can post comments on exhibits to discuss failure cases with others. The commenting system supports nested replies, allowing for threaded discussions.

- Content management: Curators can create and edit exhibits through Django's admin interface. All changes are logged in the admin log table, creating an audit trail for accountability. The curator workflow supports adding structured failure information, uploading images with attribution, linking supporting artefacts and documenting timeline information.

Key Benefits:

The prototype demonstrates several important benefits such as:

- Learning from mistakes: Rather than solely focusing on the success of AI, the museum provides insights that aren't usually that well documented and this allows for people to learn what can go wrong. This can help people be aware of the limitations are errors that AI can make.

- Structured Analysis: Each exhibit breaks down complex failures into understandable components which helps users develop an understanding of AI and how to evaluate it and make their own decisions.

- Testing Knowledge: The quizzes allows the user to turn the reading into learning by allowing the users to apply their understanding to answer the questions with immediate feedback with explanations that reinforces learning.

Key Risks:

- Content Accuracy: There's potential for AI failure case studies to oversimplify complex technical issues, get facts wrong or unintentionally damage reputations.

- Unmoderated comments: The commenting system currently has no moderation - which means that anyone can post anything.

Project description and prioritised requirements

Original Problem:

AI systems are becoming increasingly prominent in many in industries and when these systems fail there can be severe consequences. However, these failures are rarely documented and people can be unaware of the effects that that can arise as a result. There is a lot more information about the success of AI and how it can be used to be beneficial which it undoubtedly true but there is little to no coverage on the potential impacts it can have. This creates a dangerous knowledge gap as people may be unprepared to recognise potential warning signs or understand common failures of AI. Without the knowledge of what went wrong and why, the same mistakes may happen again and again.

Solution:

Our AI Failures Museum addresses this gap by creating a digital repository of AI failure cases. Focusing exclusively on failures, limitations and the misuses of AI-supported systems. Each exhibit presents a structured breakdown of a specific failure, including the system's intended purpose, what went wrong, contributing factors, and actionable lessons learned.

The system has 3 primary user groups:

- **Visitors**: University students, professionals and researchers who browse exhibits, complete quizzes and learn from the documented failures.

- **Curators**: Experts who create and edit exhibits through Django's admin interface.

- **Developers**: Our team members who access the GitHub repository, run the system locally and deploy updates.

Scope Boundaries

In scope for Sprint 1 (Currently implemented)

- Exhibit browsing: Users can view a list of all exhibits and click through to detailed pages showing structured failure information

- Interactive quizzes: Each exhibit has multiple-choice quiz questions. The system randomises answer order, processes submissions, calculates scores with percentages and displays immediate feedback with colour-coded results and explanations for every question.

- Content management: Curators can create and edit exhibits via Django admin, with all changes logged for audit trails

- Role-based Access: Three user types (visitors, curators, admins) with appropriate permissions enforced by Django.

- Comment system: Visitors can post comments on exhibits to discuss failure cases.

- Quiz scores: stores users best score per exhibit in the database

Out of scope for Sprint 1 (Not implemented)

- Comment moderation: Comments post immediately without approval. No flagging system for users to report inappropriate content.

- Analytics: No tracking of which exhibits are most viewed, which quizzes have lowest accuracy or other usage metrics for curators.

Permanent Out of Scope:

- User-generated exhibits: Only curators can create exhibits. Visitors cannot submit their own failure stories to prevent misinformation.

- Social networking features: No friend connections or social sharing buttons to maintain focus on education.

- Commercial monetisation

Success Criteria

We have 4 areas of success for our Sprint 1 success criteria:

1. Complete functions within the Museum

- At least 10 exhibits covering multiple domains
- At least 30 quiz questions
- User authentication system
- Quiz score tracking

2. Technical Quality

- Database integrity
- Security basics
- Version control

3. Content Quality

- Exhibits cover diverse failure types
- Exhibits cover multiple sectors and industries
- Quiz questions test understanding

4. Usability

- Clear navigation
- User friendly design

Requirements as GitHub Issues

Our requirements are tracked as user stories prioritised using MoSCoW (Must/Should/Couldn't/Won't). With each story having story points reflecting the implementation complexity.

Must-Have Features (Sprint 1 - All Implemented):

Feature 1: Login and comments

User Story: As a researcher, I want to be able to login so that I can leave comments

MoSCoW: Must

Story Point: 6

Feature 2: Quiz System

User Story: As a student, I want to be able to do the quizzes so that I can test my knowledge

MoSCoW: Must

Story Point: 3

Feature 3: View Historical AI Failures

User Story: As an AI researcher, I want to see what AI was like 6 months ago compared to now so that I can learn from previous AI's mistakes

MoSCoW: Must

Story Point: 0.5

Feature 4: Safe and Efficient Website

User Story: As a general user, I want to be able to have a functioning website which is safe and efficient

MoSCoW: Must

Story Point: 5

Should-Have Features (Sprint 2 - Planned):

Feature 1: Create New Cases

User Story: As a journalist, I want to be able to edit and create new cases so that other people can view them

MoSCoW: Should

Story Point: 5

Feature 2: View Changes for Moderation

User Story: As a moderator, I want to be able to view changes so that the website can be regulated

MoSCoW: Should

Story Point: 2

Feature 3: View Images and Links

User Story: As a student, I want to be able to view images or links about the cases

MoSCoW: Should

Story Point: 2

Could-Have Features (Future Enhancements):

Feature 1: Annotate Exhibits

User Story: As a general user, I want to be able to annotate an exhibit so that I can make notes

MoSCoW: Could

Story Point: 3

Feature 2: Exhibit Bookmarking

User Story: As a general user, I want to be able to bookmark exhibits so that I can have a personal favourite reading list

MoSCoW: Could

Story Point: 3

Feature 3: Mobile app

User Story: As a general user, I want to have a mobile app where I can use an offline mode and download off the app store

MoSCoW: Could

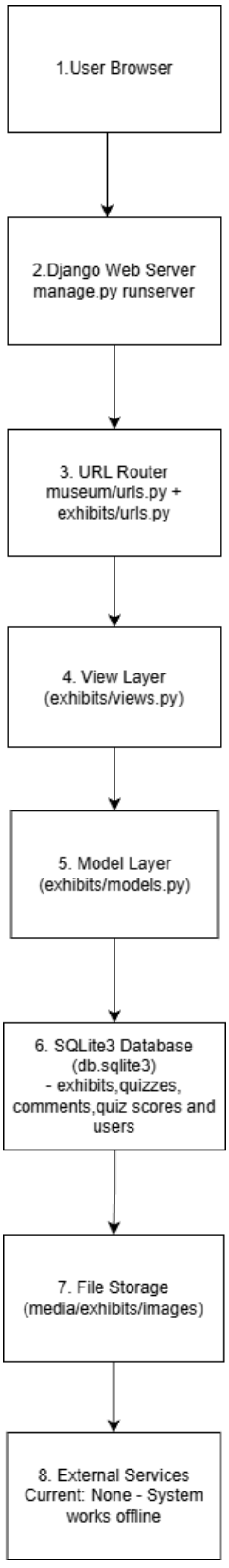
Story Point: 15

Won't Have:

- User-generated content without curator review
- Social networking features
- Commercial monetisation

Architecture (v1)

High-Level Component Diagram:



1. User Browser

The web browser that visitors use to access and view the AI Failures Museum website.

2. Django Web Server

The application server that receives HTTP requests from users and sends back HTML responses with the requested content.

3. URL Router

The component that maps incoming URLs to the appropriate view function, directing /exhibits/5/ to show exhibit 5 or /quiz/3/ to display quiz 3.

4. View Layer

Python functions in views.py that process user requests, retrieve data from models and render the appropriate response for each page.

5. Model Layer

Python classes that define the database structure for Exhibit, Quiz, Comment and QuizScore, providing an interface to interact with data.

6. SQLite3 Database

The database file that stores all persistent data including exhibits, quizzes, comments, quiz scores and user accounts.

7. File Storage

The directory where curator-uploaded exhibit images are stored, separate from the main database.

8. External Services

Current: None - the system operates completely offline.

Data Model and Dataset Description

Database Schema: Our database uses a relational structure with three main content tables also Django's built in authentification and logging tables. All data is stored in a single SQLite3 file.

Core Entities

Exhibit Table (exhibits_exhibit):

This is our primary entity representing an AI failure case study. Each exhibit has 21 fields:

Basic Information:

- id: Primary Key
- title: case study name
- domain: Sector category
- created_at: timestamp

System Context:

- deployment_context: Where and why the system was deployed
- intended_use: What the system was supposed to accomplish
- system_type: Type of AI/ML system in the case study

Technical Details:

- inputs_and_assumptions: Data inputs and underlying assumptions
- outputs_presented: What the system produced

Failure Documentation:

- failure_description: What went wrong
- detection_method: How the failure was discovered
- affected_parties: Who was impacted

Analysis:

- contributing_factors: General factors

- data_issues: Data-related problems
- technical_choices: Technical decisions
- organizational_factors: Governance issues
- lessons_learned: What can be learned
- timeline: Chronology of events

Supporting Materials:
- image: Visual representation
- image_reference: Attribution
- supporting_artefacts: Source links

Quiz Table (exhibits_quiz):

Multiple-choice questions linked to exhibits. This entity has 6 fields:

- id: Primary key
- exhibit_id: Foreign key to Exhibit
- question: Question text
- options: Answer choices
- correct_answer_index: Which option is correct
- explanation: Why answer is correct

Comment Table (exhibits_comment):

User-posted discussions with nested reply support. This entity has 7 fields:

- id - Primary key
- exhibit_id: Foreign key to Exhibit
- parent_id: Foreign key to Comment
- user_id: Foreign key to user
- author_name: Display name
- body: Comment text
- created_at: Timestamp

QuizScore Table (exhibits_quizscore):

Stores users' best quiz performance per exhibit. This entity has 7 fields:

- id: Primary key
- user_id: Foreign key to User
- exhibit_id: Foreign key to Exhibit
- correct_answers: Integer
- total_questions: Integer
- score_percentage: Real number
- taken_at: Timestamp

Supporting Tables (Django Built-in)

User Authentication (auth_user): Django's standard User model with key fields

- username: Unique identifier
- email: Email address
- password: Hash string
- first_name, last_name: Optional
- is_staff: curator access
- is_superuser: admin access
- is_active: disable accounts
- date_joined, last_login : Timestamps

Session Management (django_session): Django's session table

- session_key: 40 character random string
- session_data: Encrypted session state
- expire_date: Sessions expire after 2 weeks

Admin Log (django_admin_log):

Django automatically records all admin actions:

- user_id: Which admin made the change
- content_type_id: Which model was affected
- object_id: Which record
- action_flag: Type: 1=add, 2=change, 3=delete
- change_message: Text description
- action_time: When the change occurred

Database Relationships

Exhibit -> Quiz (One-To_Many): Each quiz belongs to exactly one exhibit and if the exhibit is deleted, then quizzes are automatically deleted.

Exhibit -> Comment (One-to-Many): Each comment is about exactly one exhibit and if the exhibit is deleted the the comments will delete too.

Comment -> Comment (Self-referential): Comments can be replies to another comment so if the original comment was deleted the reply to that comment would be deleted.

Seed Data Files:

We maintain our dataset as JSON files with the three files:

-exhibits.json: This stores the exhibits

-quizzes.json: This stores the quizzes

-artefacts.json: This stores the supporting artefacts and source links

Data Curation Process:

Each exhibit was manually researched by team members and followed this structured approach:

1.Identify publicly documented AI failures

2.Gather information from credible sources

3.Structure into schema

4.Write 3 quiz questions per exhibit

5.Peer review for accuracy

6.Add to JSON seed files

Data Quality Standards:

- Source attribution: supporting_artefacts field contains credible source URLs
- Peer reviewed: All exhibits were checked by other team members
- Educational focus: Content emphasises lessons learned
- Complete fields: All required fields are populated

Evaluation plan (initial):

Implemented Measure: Quiz Comprehension Accuracy

Our primary measure is quiz comprehension accuracy, tracked through our QuizScore system which stores each users best score per exhibit in the database.

Early Results:

From our initial testing, we have 2 quiz attempts stored from 1 user:

- Exhibit 16: 3/3 correct (100%)

- Exhibit 17: 2/3 correct (66.7%)

- Average score: 83.3%

The variation between exhibits suggests quizzes differ in difficulty.

We also developed an automated test suite of 28 tests covering authentication, comments, quiz scoring, and page loading, all of which are included in the repository under exhibits/tests/.

What We Will Evaluate in CW2:

1. Quiz comprehension with a lot more participants across all exhibits to generate meaningful accuracy data and identify which domains are harder or easier.

2. Content accuracy audit with review of all exhibits to verify factual correctness and credibility of sources.

3. How effective the badge system is on our home page as for each exhibit a bronze, silver and gold badges is given to a user based on their quiz scores (bronze: >0%, silver: 50%+, gold: 80%+). We will evaluate whether these badges motivate users to retake quizzes and improve their scores over time.

Risk register:

Risk Classification

Risks are classified by:

- Likelihood: Low/Medium/High
- Impact: Low/Medium/High/Critical

Technical Risks

1. Database corruption or loss

Likelihood: Low

Impact: Critical

Risk Score: High

Description: The SQLite file corrupts or was accidently deleted which results in loss of exhibits and quizzes.

Mitigation: Daily backups of file

Owners: Software Developer

2. Quiz Answer Shuffling Bug

Likelihood: Low

Impact: Medium

Risk Score: Low

Description: Wrong answers being marked as correct

Mitigation: Ensure quizzes are fully tested to catch bugs

Owners: Software Developer

3. Comment Spam Flood

Likelihood: Medium

Impact: Medium

Risk Score: Medium

Description: User posts hundreds of comments

Mitigation: Limit amount of comments by a user by a time period

Owners: Software Developer

Delivery Risks

1. Team Member Unavailability

Likelihood: Medium

Impact: High

Risk Score: High

Description: Key developer unavailable during the sprint

Mitigation: Code documentation with comments so that other team members understand the code and could also have weekly knowledge transfer sessions

Owners: Scrum Master

2. Integration Issues

Likelihood: Medium

Impact: Medium

Risk Score: Medium

Description: Frontend and backend changes dont work together

Mitigation: Integration tests

Owners: Testing

3. Misunderstanding Requirements

Likelihood: Low

Impact: Critical

Risk Score: High

Description: Building features that don't match the specification

Mitigation: Weekly specification review to ensure that the project remains on track

Owners: Documentation

Operational Risks

1. Misinformation in Exhibits

Likelihood: Medium

Impact: Medium

Risk Score: Medium

Description: Factual errors that can cause users to learn incorrect information

Mitigation: Credible sources and peer review for approval

Owners: Data Lead

2. Inappropriate Comment Content

Likelihood: Medium

Impact: Low

Risk Score: Medium

Description: Users post offensive/spam content

Mitigation: Manual deletion by admin

Owners: Scrum Master

3. Quiz Difficulty Imbalance

Likelihood: Low

Impact: Low

Risk Score: Low

Description: Quizzes too easy or too hard

Mitigation: Detailed explanations and unlimited retakes

Owners: Data Lead